



Trace the Bug

Ontario Math C3.2 — read, predict, and fix code

Name:

Date:

★ I can trace a loop step by step to find the bug.

Walk through the code

To find a bug, TRACE the code: go through the loop one repeat at a time and keep track of what Bit has done.

Trace the loop below. After all the repeats, did Bit do what it should? If not, you have found the bug.

Trace this — it **SHOULD** draw a square

```
for i in range(3):  
    bit.forward(50)  
    bit.right(90)
```

PYTHON

OUTPUT

Trace: side 1, side 2, side 3 ... then it stops.

1 Trace it

Go through the loop. After repeat 1: 1 side. After repeat 2: 2 sides. After repeat 3: _____ sides. A square needs 4 sides — what is the bug, and the fix?

2 Trace the counts

A loop is repeat 4 { jump, jump }. Trace the jumps: after each repeat add 2. How many jumps in total? If Bit should only jump 4 times, what is the fix?

3 Predict, then run

Trace for `i in range(5): bit.forward(50); bit.right(90)`. How many sides does Bit draw? Is that right for a square? What is the fix?

4 Challenge

Trace for `i in range(4): bit.forward(50); bit.right(60)`. It has 4 sides but the shape does not close. The range is right — so what is the bug?

Big idea

Tracing means walking through the code one repeat at a time. If the end result is not the goal, the place where it goes wrong is the bug.



Trace the Bug

Quick facilitation notes & answer key

What this worksheet teaches

Students TRACE a loop one repeat at a time to find exactly where it goes wrong, then name the fix — covering wrong count, too many, and a wrong turn. Tracing is the careful way to find a bug. No coding experience needed.

Before you start

No computer needed — the worksheet shows the code and what Bit should do. Optional: a Scratch-style app (for the block loops) or a free Python-turtle playground (for the shapes) to run the code — you only press Run.

How to lead it (about 30 minutes)

1. Show it (you do). Go through the loop one repeat at a time and watch the running total.
2. Do one together. Exercise 1 — after 3 repeats Bit has 3 sides; a square needs 4.
3. Let them work. Students trace a jump count (Ex 2), a square with too many sides (Ex 3), and a square with a wrong turn (Ex 4).

Answer key (these are the verified correct answers)

Exercise 1 — 3 sides; the bug is the repeat number; fix to repeat 4. Exercise 2 — 8 jumps; fix to repeat 2 ($2 \times 2 = 4$). Exercise 3 — 5 sides (one too many); fix to repeat 4. Exercise 4 (challenge) — the turn is wrong (right 60); a square needs turn right 90.

If students get stuck — and ways to adjust

Watch for: guessing without tracing. Go repeat by repeat and write the running total.

Make it easier: make a little table — repeat 1, 2, 3... and the total after each.

Make it harder: ask them to trace a six-sided shape and find its bug.

Ontario curriculum link (official wording)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential, concurrent, and repeating events.

In plain terms: students write and run step-by-step instructions — including a "repeat" instruction — to make something happen.

C3.2 — read and alter existing code, including code that involves sequential, concurrent, and repeating events, and describe how changes to the code affect the outcomes.

In plain terms: students read instructions, find what is wrong, fix it, and explain what the change did.

You'll know it worked when

The child traces the loop repeat by repeat, finds where it goes wrong, and names the fix.